USN NO- CS23O82

Mrunal Samrutwar

Extra Dataset Codes Based On practical 3

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from google.colab import files
files.upload()
```

```
data = pd.read_csv("diabetes_prediction_dataset.csv")
print(data.head())
```

```
   gender   age  hypertension  heart_disease smoking_history    bmi  \
0  Female  80.0             0              1           never  25.19
1  Female  54.0             0              0        No Info  27.32
2    Male  28.0             0              0           never  27.32
3  Female  36.0             0              0         current  23.45
4    Male  76.0             1              1         current  20.14

   HbA1c_level  blood_glucose_level  diabetes
0          6.6                  140         0
1          6.6                   80         0
2          5.7                  158         0
3          5.0                  155         0
4          4.8                  155         0
```

```
print(data.info())
print(data.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
None
                 age  hypertension  heart_disease           bmi  \
count  100000.000000  100000.00000  100000.000000  100000.000000
mean       41.885856       0.07485       0.039420      27.320767
std        22.516840       0.26315       0.194593       6.636783
min         0.080000       0.00000       0.000000      10.010000
25%        24.000000       0.00000       0.000000      23.630000
50%        43.000000       0.00000       0.000000      27.320000
75%        60.000000       0.00000       0.000000      29.580000
max        80.000000       1.00000       1.000000      95.690000

         HbA1c_level  blood_glucose_level       diabetes
count  100000.000000        100000.000000  100000.000000
mean       5.527507           138.058060       0.085000
```

```
std       1.070672      40.708136      0.278883
min       3.500000      80.000000      0.000000
25%       4.800000     100.000000      0.000000
50%       5.800000     140.000000      0.000000
75%       6.200000     159.000000      0.000000
max       9.000000     300.000000      1.000000
```

```
print(data.isnull().sum())
```

```
gender               0
age                  0
hypertension         0
heart_disease        0
smoking_history      0
bmi                  0
HbA1c_level          0
blood_glucose_level  0
diabetes             0
dtype: int64
```

```
data_encoded = pd.get_dummies(data, drop_first=True)
print(data_encoded.head())
```

```
   age  hypertension  heart_disease   bmi  HbA1c_level  blood_glucose_level  \
0  80.0             0              1  25.19          6.6                  140
1  54.0             0              0  27.32          6.6                   80
2  28.0             0              0  27.32          5.7                  158
3  36.0             0              0  23.45          5.0                  155
4  76.0             1              1  20.14          4.8                  155

   diabetes  gender_Male  gender_Other  smoking_history_current  \
0         0        False         False                    False
1         0        False         False                    False
2         0         True         False                    False
3         0        False         False                     True
4         0         True         False                     True

   smoking_history_ever  smoking_history_former  smoking_history_never  \
0                 False                   False                   True
1                 False                   False                  False
2                 False                   False                   True
3                 False                   False                  False
4                 False                   False                  False

   smoking_history_not current
0                        False
1                        False
2                        False
3                        False
4                        False
```

```
X = data_encoded.drop("diabetes", axis=1)
y = data_encoded["diabetes"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.95905
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[18126   166]
 [  653  1055]]
```

```
print("Classification Report:\n")
print(classification_report(y_test, y_pred))
```

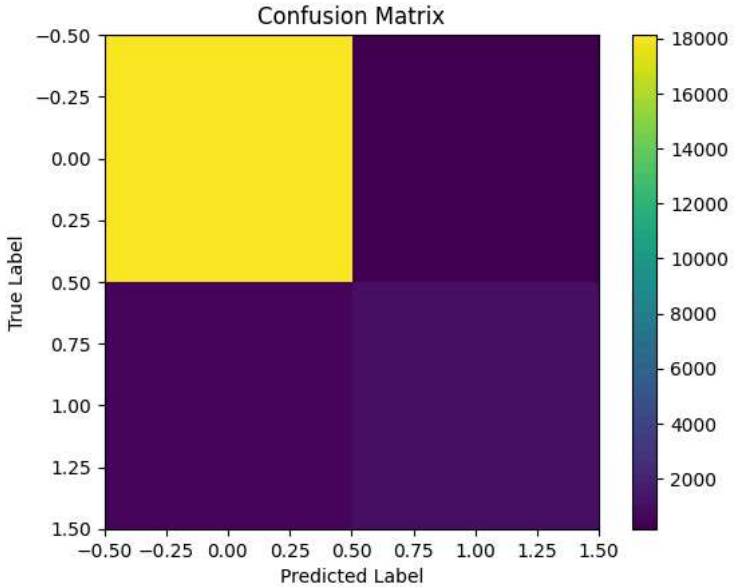
```
Classification Report:

              precision    recall  f1-score   support

           0       0.97      0.99      0.98     18292
           1       0.86      0.62      0.72      1708

    accuracy                           0.96     20000
   macro avg       0.91      0.80      0.85     20000
weighted avg       0.96      0.96      0.96     20000
```

```
plt.imshow(cm)
plt.colorbar()
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```




```
new_data = X.iloc[:1]
prediction = model.predict(new_data)
```

```
print("Predicted Class (0 = No Diabetes, 1 = Diabetes):", prediction)
```

```
Predicted Class (0 = No Diabetes, 1 = Diabetes): [0]
```

```
probability = model.predict_proba(new_data)
print("Prediction Probabilities:", probability)
```

```
Prediction Probabilities: [[0.60941096 0.39058904]]
```